* After each source statement is processed, the length of the assembled inst. is added with LOCCTR (Eg) RESW 10

10×3 - bytes will be added to LOCCTR

LOCCT 1020

RESB 5 1025

8100 8102

00 81 06

3A RESB 5

Machine dependant Assembler features

There are two features

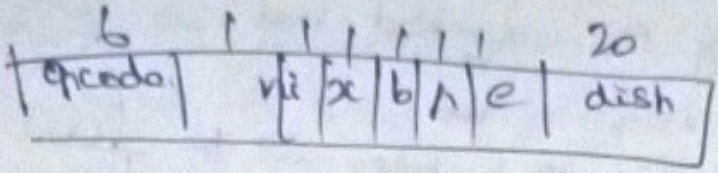
* Instruction formats and addressing mode
* Relocation

Inst formats & addressing modes

As discussed in SIC/XE there are 4 inst formats

1. Format 1 (1byte) — 8 bit: opcode — Eg: RSUB, JSUB
2. Format 2 (2bytes) — 8 | 4 | 4: opcode | n | r2 — (Eg) CLEAR X, A
3. Format 3 (3bytes) — 6 | 1 | 1 | 1 | 1 | 1 | 1 | 12: opcode | n | i | x | b | p | e | displacement

4. Format (4 bytes)

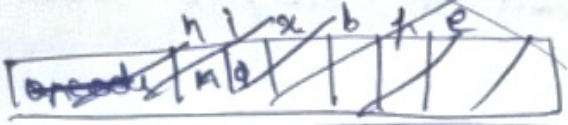


Addressing modes

displacement value should be $-2048 \leq disp \leq 2047$

1. Base relative — TA = [B] + disp — $0 \leq disp \leq 4095$
2. PC relative — TA = [PC] + disp — $-2048 \leq disp \leq 2048$
3.
4. Indirect addressing → @ — @LABEL
5. Immediate addressing → # — LDA #3
6. Index addressing → X (Eg) BUFFER, X

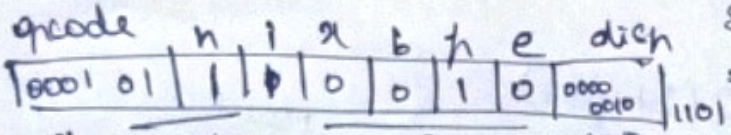
(Eg) 0000 FIRST STL RETADR format 4 will ↑



(Eg) 0000 FIRST STL RETADR - It is not format 4 since it will have ↑ addressing

PC = 0003

our program starts at 0000 & format 2 so PC = 0003



TA = [PC] + disp

0030 = 0003 + disp

disp = 0030 − 0003 = 02D

17202D - First Inst.

RETADR 0030

	hex	dec
disp = 0030	20 →	48
0003	3	3
0027		48 - 3 = 45 ↓ 2D

STL 14

Relocation

* Different programers write programs each programer will specify an address & two programmer specifies same address

* Program cannot be runned when they use same address

Eg: 55 101B LDA THREE 0012D

1000 → 1020 THREE is present

2000

&

TWO

(Eg) → 0006 CLOOP +JSUB RDREC 4B101036

Last 5 bit address in Format 4

0013 +JSUB WREC 4B10105D

starts at 0006 ends at 0009

20 bits for disp divided into halfs the

20/4 u = 05

0006 JSUB

0007

0008

0009

Modification Record

Col M

col 2-7 Starting location of the address field to be modified relative to the beginning of the program (hexadecimal)

col 8-9 Length of the address field to be modified in halfbytes (hexadecimal)
4 byte

M^000007^05

0013 +JSUB WRREC
0014
15
16

M^000014^05

Machine Independent Assembler features

1. Literals
2. Symbol defining statements
3. Expressions
4. Program blocks
5. Control sections & program linking

Literals

* writting a value of constant operand as a part of the inst. such operands are called literals

value ↓ ↙ End of file

Eg LDA =C'EOF' — 3 byte value

LDY INPUT =X'05' — 1 byte

* Literals are identified by the prefix '='

In immediate addressing (Eg) LDA #4096 the constant is part of the inst. In literals value is assigned to some string.

* All literals used in the program are gathered together into one or more literal pools.

* All literals are used in the End of the program

END

= } literal pool.

* when the literals are more we use LTORG

LTORG

=C'EOF

LTORG Assembler Directive

* Sometimes literals are placed into a pool at some other location in the object program. other than end of the program

* For this purpose we use new assembler directive LTORG

* When LTORG is used the assembler creates a literal pool that contains all the literal operands that are used since ~~the~~ previous LTORG or begining of the program

* Literals placed using LTORG will not be ~~expanded~~ at the end of the program.

Symbol ~~definition~~ defining statements

There are two Symbol defining statements

* EQU
* ORG

EQU

Symbol EQU value (eg) MAXLEN EQU 4096

⋮

+LDT #MAXLEN

* Assembler provides an assembler directive that allows the programmer to define symbols & specify their values

* The assembler directive used is EQU

ORG

* Resets the value of the Location counter (LOCCTR)

LOCCTR - starting address of the pgm initially then increases as per inst format.

* When an ORG statement is encountered the LOCCTR value will be changed to specified address in ORG

Expressions

Types 1. Absolute - constant (constant) 10+12 = 12

2. Relative - BUFFER - BUFFEREND (symbols)
1032 - 1030
= 2 - Absolute.

~~Absolute Expression~~

* Most assemblers allow arithmetic exp which when evaluated gives the operand address of value

* Assembler allow arithmetic expressions with the operators "+", -, * & "/"

* Expressions may have user defined symbols, constants or special symbols.

Program Blocks

* Generally program being assembled was treated as a unit.

* The pgm logically contained subroutines, data areas etc. however the assembler handled them as one entity resulting in a single block of object code.

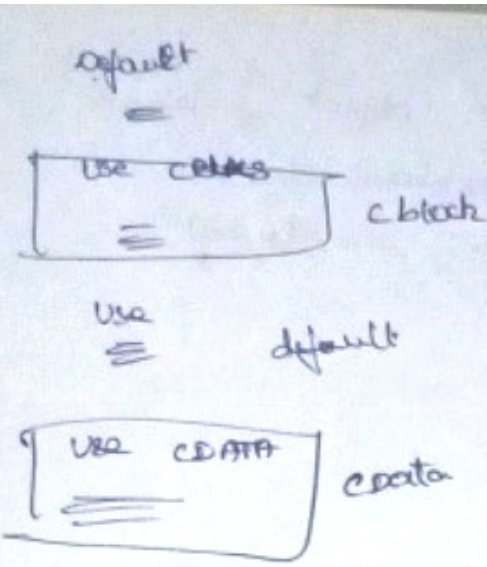
* Program block refers to segments of the code rearranged within a single object program unit.

* 2 blocks are used to write programs

* Default block : executable inst.

* CDATA block : data areas with length

* CBLKS : all data with larger block of memory.



control section & program linking

Control section refers to segments that are translated into independent object program units.

* Content of control unit section will not change even after assembly.

Two types control section

* EXTDEF (external Definition) : cl used in C2 C3
* EXTREF (external Reference) used C1 define in C2 C3

Generally in object program format we use 3 types of records

1. Header; contains program name, starting addr & length.
2. Text: contains translated inst and data of program together with an indication of the addresses where these are to be loaded.

End : Marks the end of the object program and specifies the address in the program where execution is to begin.

Header record

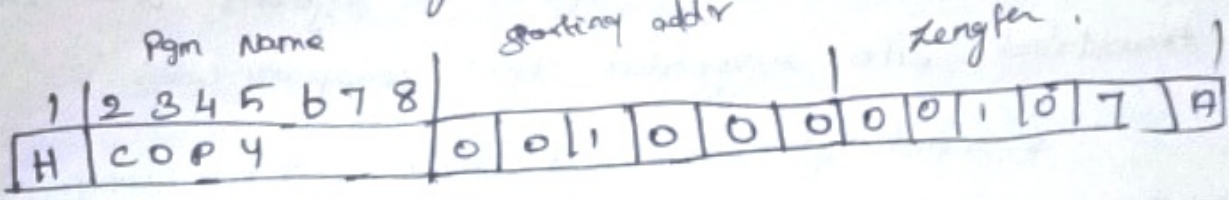
Col 1 : H

Col 2-7 : Pgm Name

Col 8-13 : starting address of obj pgm (hexadecimal)

Col 14-19 : length of obj pgm in byte (")

Eg.



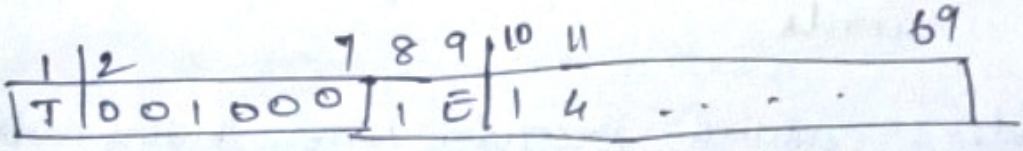
Text record

Col 1 : T

Col 2-7 : Starting address of obj code in this record

Col 8-9 : length

Col 10-69 : object code



End record

Col 1 : E

Col 2-7 : Address of first executable inst.

Eg: E 001000

Define Record

Col 1 : D

Col 2-7 : Name of the external symbol defined in this control section

Col 8-13 Relative address of symbol within this control section

Col 14-73 Repeat info of col 2-13 for other external symbols

(Eg)

D^BUFFER, 000033^BUFFEREND^C0L033

Refer record

Col 1 : R

Col 2-7 : Name of external symbol referred in this control section

Col 8-73 : Name of external records used

Eg R^RDREC, WRREC